

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-210БВ-25

Студент: Соколов И.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 09.10.26

Москва, 2026

Постановка задачи

Вариант 4.

Разработать программу на языке Си, реализующую взаимодействие между двумя процессами (родительским и дочерним) с использованием неименованных каналов (pipe):

1. Родительский процесс считывает из стандартного потока ввода имя выходного файла и передает его дочернему процессу.
2. Далее родительский процесс считывает строки, содержащие вещественные числа, и построчно пересылает их через канал в дочерний процесс.
3. Дочерний процесс принимает данные из канала и выполняет операцию последовательного деления чисел в каждой строке: первое число делится на второе, промежуточный результат — на третье и так далее.
4. Результаты вычислений дочерний процесс записывает в указанный выходной файл.
5. При попытке деления на ноль дочерний процесс должен корректно обработать исключительную ситуацию, вывести диагностическое сообщение об ошибке и завершить работу.
6. Родительский процесс ожидает завершения выполнения дочернего процесса и корректно освобождает системные ресурсы перед завершением работы.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; — создает дочерний процесс путем создания точной копии адресного пространства родительского процесса.
- `int pipe(int pipefd[2])`; — создает однонаправленный канал передачи данных между процессами (дескриптор `pipefd[0]` используется для чтения, `pipefd[1]` — для записи).
- `int dup2(int oldfd, int newfd)`; — дублирует файловый дескриптор, позволяя перенаправить стандартный поток ввода `stdin` на дескриптор чтения из канала.
- `int execl(const char *path, const char *arg0, ...)`; — заменяет текущий образ дочернего процесса исполняемым файлом `child`.
- `pid_t waitpid(pid_t pid, int *status, int options)`; — блокирует родительский процесс до момента завершения работы дочернего процесса и получает его статус завершения.
- `int close(int fd)`; — закрывает неиспользуемые дескрипторы каналов, что необходимо для корректной передачи сигнала завершения ввода (EOF).
- `int open(const char *pathname, int flags, mode_t mode)`; — открывает или создает выходной файл для записи результатов вычислений.
- `ssize_t read(int fd, void *buf, size_t count)`; — осуществляет низкоуровневое чтение данных из канала или потока.
- `ssize_t write(int fd, const void *buf, size_t count)`; — осуществляет запись данных в канал или файл.

Описание алгоритма работы программы:

1. Родительский процесс с помощью вызова `pipe()` создает неименованный канал для передачи данных дочернему процессу.
2. Системным вызовом `fork()` создается дочерний процесс.
3. В родительском процессе закрывается дескриптор канала на чтение. Со стандартного потока ввода считывается имя результирующего файла, после чего оно передается дочернему процессу.
4. В дочернем процессе закрывается дескриптор канала на запись, а дескриптор чтения перенаправляется на стандартный поток ввода (`stdin`) с помощью `dup2()`. Затем вызывается семейство функций `exec` для загрузки бинарного кода программы `child`.
5. Программа `child` открывает целевой файл на запись с флагами создания и перезаписи (`O_WRONLY | O_CREAT | O_TRUNC`).
6. Родительский процесс считывает строки с числами из стандартного ввода и построчно отправляет их в канал. По окончании ввода (признак EOF) дескриптор записи закрывается.
7. Дочерний процесс считывает строки, парсит вещественные числа и выполняет их последовательное деление (первое число делится на второе, результат — на третье и так далее).
8. В случае обнаружения делителя, равного нулю, дочерний процесс выводит диагностическое сообщение об ошибке деления на ноль в поток `stderr` и аварийно завершает работу.
9. При корректных данных результаты деления записываются в выходной файл.
10. Родительский процесс с помощью `waitpid()` ожидает завершения дочернего процесса, освобождает занятые ресурсы и завершает работу.

Код программы

parent.c

```
#include <stdint.h>

#include <stdbool.h>

#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <fcntl.h>

static char SERVER_PROGRAM_NAME[] = "child";

static uint32_t str_length(const char *s) {
    uint32_t len = 0;
    while (s[len] != '\0') {
        ++len;
    }
    return len;
}

static void write_all(int32_t fd, const char *buf, size_t size) {
    size_t written = 0;
```

```

        while (written < size) {
            ssize_t res = write(fd, buf + written, size - written);
            if (res <= 0) {
                break;
            }
            written += (size_t)res;
        }
    }

static ssize_t read_line(int32_t fd, char *buf, size_t max_size) {
    size_t idx = 0;
    char ch;
    while (idx + 1 < max_size) {
        ssize_t bytes = read(fd, &ch, 1);
        if (bytes <= 0) {
            if (idx == 0) {
                return -1;
            }
            break;
        }
        if (ch == '\r') {
            continue;
        }
        if (ch == '\n') {
            break;
        }
        buf[idx++] = ch;
    }
    buf[idx] = '\0';
    return (ssize_t)idx;
}

int main(int argc, char **argv) {
    char filename[1024];

    if (argc > 1) {
        uint32_t i = 0;
        while (argv[1][i] != '\0' && i < sizeof(filename) - 1) {
            filename[i] = argv[1][i];
            ++i;
        }
        filename[i] = '\0';
    } else {
        const char prompt[] = "Enter output filename: ";
        write_all(STDOUT_FILENO, prompt, sizeof(prompt) - 1);

        ssize_t len = read_line(STDIN_FILENO, filename, sizeof(filename));
        if (len <= 0) {
            const char msg[] = "error: invalid filename\n";

```

```

        write_all(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
}

char prospath[1024];
{
    ssize_t len = readlink("/proc/self/exe", prospath, sizeof(prospath) - 1);
    if (len <= 0) {
        prospath[0] = '.';
        prospath[1] = '\0';
    } else {
        while (len > 0 && prospath[len] != '/') {
            --len;
        }
        prospath[len] = '\0';
    }
}

// NOTE: Open pipes
int client_to_server[2];
if (pipe(client_to_server) == -1) {
    const char msg[] = "error: failed to create pipe\n";
    write_all(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

int server_to_client[2];
if (pipe(server_to_client) == -1) {
    const char msg[] = "error: failed to create pipe\n";
    write_all(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

const pid_t child = fork();

switch (child) {
case -1: {
    const char msg[] = "error: failed to spawn new process\n";
    write_all(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
} break;

case 0: {
    close(client_to_server[1]);
    close(server_to_client[0]);

    if (dup2(client_to_server[0], STDIN_FILENO) == -1) {
        _exit(EXIT_FAILURE);
    }
}
}

```

```

    }
    close(client_to_server[0]);

    if (dup2(server_to_client[1], STDOUT_FILENO) == -1) {
        _exit(EXIT_FAILURE);
    }
    close(server_to_client[1]);

    {
        char path[1024];
        uint32_t p_len = str_length(progpath);
        uint32_t s_len = str_length(SERVER_PROGRAM_NAME);

        for (uint32_t i = 0; i < p_len; ++i) {
            path[i] = progpath[i];
        }
        path[p_len] = '/';
        for (uint32_t i = 0; i < s_len; ++i) {
            path[p_len + 1 + i] = SERVER_PROGRAM_NAME[i];
        }
        path[p_len + 1 + s_len] = '\0';

        char *const args[] = {SERVER_PROGRAM_NAME, filename, NULL};
        int32_t status = execv(path, args);

        if (status == -1) {
            const char msg[] = "error: failed to exec into child
executable\n";
            write_all(STDERR_FILENO, msg, sizeof(msg) - 1);
            _exit(EXIT_FAILURE);
        }
    }
    } break;

default: {
    close(client_to_server[0]);
    close(server_to_client[1]);

    const char prompt[] = "Enter float numbers (empty line to exit):\n";
    write_all(STDOUT_FILENO, prompt, sizeof(prompt) - 1);

    char buf[4096];
    ssize_t bytes;

    while ((bytes = read_line(STDIN_FILENO, buf, sizeof(buf))) >= 0) {
        if (buf[0] == '\0') {
            break;
        }
    }
}

```

```

        write_all(client_to_server[1], buf, (size_t)bytes);
        write_all(client_to_server[1], "\n", 1);

        char status_byte = 0;
        ssize_t res = read(server_to_client[0], &status_byte, 1);

        if (res <= 0 || status_byte == 'E') {
            const char msg[] = "error: division by zero detected!
Terminating.\n";
            write_all(STDERR_FILENO, msg, sizeof(msg) - 1);
            break;
        }

        close(client_to_server[1]);
        close(server_to_client[0]);

        wait(NULL);
    } break;
}

return 0;
}

```

child.c

```

#include <stdint.h>
#include <stdbool.h>

#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

static void write_all(int32_t fd, const char *buf, size_t size) {
    size_t written = 0;
    while (written < size) {
        ssize_t res = write(fd, buf + written, size - written);
        if (res <= 0) {
            break;
        }
        written += (size_t)res;
    }
}

static void float_to_str(float val, char *buf) {
    if (val < 0.0f) {
        *buf++ = '-';
        val = -val;
    }
}

```

```

int64_t int_part = (int64_t)val;
float frac = val - (float)int_part;

char temp[32];
int32_t temp_idx = 0;

if (int_part == 0) {
    temp[temp_idx++] = '0';
} else {
    while (int_part > 0) {
        temp[temp_idx++] = (char)('0' + (int_part % 10));
        int_part /= 10;
    }
}

while (temp_idx > 0) {
    *buf++ = temp[--temp_idx];
}

*buf++ = '.';

for (int32_t i = 0; i < 6; ++i) {
    frac *= 10.0f;
    int32_t digit = (int32_t)frac;
    if (digit > 9) {
        digit = 9;
    }
    *buf++ = (char)('0' + digit);
    frac -= (float)digit;
}

*buf++ = '\n';
*buf = '\0';
}

static ssize_t read_line(int32_t fd, char *buf, size_t max_size) {
    size_t idx = 0;
    char ch;
    while (idx + 1 < max_size) {
        ssize_t bytes = read(fd, &ch, 1);
        if (bytes <= 0) {
            if (idx == 0) {
                return -1;
            }
            break;
        }
        if (ch == '\r') {
            continue;
        }
    }
}

```

```

        if (ch == '\n') {
            break;
        }
        buf[idx++] = ch;
    }
    buf[idx] = '\0';
    return (ssize_t)idx;
}

int main(int argc, char **argv) {
    if (argc < 2) {
        const char msg[] = "error: child requires filename\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    int32_t file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC | O_APPEND, 0600);
    if (file == -1) {
        const char msg[] = "error: failed to open requested file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        write(STDOUT_FILENO, "E", 1);
        exit(EXIT_FAILURE);
    }

    char buf[4096];
    ssize_t bytes;

    while ((bytes = read_line(STDIN_FILENO, buf, sizeof(buf))) >= 0) {
        char *ptr = buf;
        char *endptr = NULL;
        float result = 0.0f;
        int32_t count = 0;
        bool div_by_zero = false;

        while (*ptr != '\0') {
            while (*ptr == ' ' || *ptr == '\t') {
                ++ptr;
            }
            if (*ptr == '\0') {
                break;
            }

            float val = strtod(ptr, &endptr);
            if (ptr == endptr) {
                break;
            }

            if (count == 0) {
                result = val;
            }
        }
    }
}

```

```

        } else {
            if (val == 0.0f) {
                div_by_zero = true;
                break;
            }
            result /= val;
        }
        ++count;
        ptr = endptr;
    }

    if (div_by_zero) {
        write(STDOUT_FILENO, "E", 1);
        close(file);
        exit(EXIT_FAILURE);
    }

    if (count > 0) {

        char out_str[64];
        float_to_str(result, out_str);

        uint32_t len = 0;
        while (out_str[len] != '\0') {
            ++len;
        }
        write_all(file, out_str, len);
    }

    write(STDOUT_FILENO, "K", 1);
}

close(file);
return 0;
}

```

Протокол работы программы

Тест 1. Корректные вычисления и запись в res.txt

```
iv4nslv@MacBook-Air--jklptmr ~ % cd ~/Desktop/"806 кафедра"/"2 курс"/"Операционные системы"/labwork_1
make

gcc -std=c11 -Wall -Wextra -Werror -pedantic parent.c -o parent
gcc -std=c11 -Wall -Wextra -Werror -pedantic child.c -o child
iv4nslv@MacBook-Air--jklptmr ~/Desktop/806 кафедра/2 курс/Операционные системы/labwork_1 % ./parent
Enter output filename: res.txt
Enter float numbers (empty line to exit):
100 2 5
64 4 2
iv4nslv@MacBook-Air--jklptmr ~/Desktop/806 кафедра/2 курс/Операционные системы/labwork_1 % cat res.txt
10.000000
8.000000
7.750000
```

Тест 2. Обработка ошибки деления на ноль

```
iv4nslv@MacBook-Air--jklptmr ~/Desktop/806 кафедра/2 курс/Операционные системы/labwork_1 % ./parent
Enter output filename: res.txt
Enter float numbers (empty line to exit):
50 5 0 2
error: division by zero detected! Terminating.
iv4nslv@MacBook-Air--jklptmr ~/Desktop/806 кафедра/2 курс/Операционные системы/labwork_1 %
```

Вывод

В ходе выполнения лабораторной работы были освоены механизмы межпроцессного взаимодействия в ОС семейства UNIX с помощью системных вызовов `fork`, `pipe` и `dup2`. Была реализована программа, осуществляющая корректную передачу данных между процессами через неименованный канал, последовательное деление чисел и сохранение результатов в файл. В процессе работы возникла небольшая трудность со своевременным закрытием неиспользуемых дескрипторов канала, из-за чего дочерний процесс не сразу получал признак конца ввода (EOF), однако проблема была успешно решена. В итоге программа полностью отлажена и протестирована на различных наборах данных, включая корректную обработку исключительной ситуации при делении на ноль.